# Agent Phantom Recovery

# 16-module-implementation-plan.md

# Production Module Implementation Plan

Version: 1.0

Status: Production Design

---

# 1. Purpose

This document defines the implementation roadmap for every major module inside Agent Phantom Recovery.

Unlike previous documents that define architecture and specifications, this document answers:

```
What needs to be built?
In what order?
Why?
What are the dependencies?
What is the implementation strategy?
```

The goal is to provide a structured path from:

```
Architecture
        ↓
Implementation
        ↓
Production System
```

---

# 2. Implementation Philosophy

Most projects fail because they build everything simultaneously.

Agent Phantom Recovery should be built in layers.

---

### Incorrect Approach

```
Build Everything
      ↓
Integrate Everything
      ↓
Debug Everything
```

Problems:

- High complexity
- Difficult debugging
- Slow progress
- Large failure surface

---

### Correct Approach

```
Foundation
      ↓
Core Execution
      ↓
Tooling
      ↓
Memory
      ↓
Repository Intelligence
      ↓
Verification
      ↓
UI Integration
      ↓
Production Features
```

---

# 3. Implementation Roadmap

The system is divided into ten implementation phases.

```
Phase 0 → Foundation
Phase 1 → Authentication
Phase 2 → Core Backend
Phase 3 → LLM Layer
Phase 4 → Tool System
Phase 5 → Memory System
```

```
Phase 6 → Repository Intelligence
Phase 7 → Agent Execution Engine
Phase 8 → Antigravity IDE
Phase 9 → Production Hardening
```

---

# 4. Phase 0 — Foundation Layer

## Goal

Create project foundations.

---

## Deliverables

```
Monorepo Setup
Environment Configuration
CI/CD Pipeline
Database Setup
Supabase Setup
Basic Frontend
Basic Backend
```

---

## Backend Stack

```
FastAPI
Python
PostgreSQL
Supabase
Redis
```

---

## Frontend Stack

```
Next.js
TypeScript
Tailwind
shadcn/ui
```

---

## Output

A running application skeleton.

---

# 5. Phase 1 — Authentication Module

## Goal

Implement identity and access management.

---

## Modules

### Auth Service

Responsibilities:

```
Login
Logout
Session Validation
Token Refresh
```

---

### User Service

Responsibilities:

```
Profiles
Preferences
Permissions
```

---

## APIs

```
/auth/login
/auth/logout
/auth/me
```

---

### Completion Criteria

```
User can register
User can login
Protected APIs work
```

---

# 6. Phase 2 — Core Backend Infrastructure

## Goal

Build platform foundations.

---

## Modules

### Project Module

```
Projects
Workspaces
Ownership
```

---

### Repository Module

```
Repository Registration
Repository Metadata
```

---

### Task Module

```
Task Creation
Task Lifecycle
Task States
```

---

### Event Module

```
Event Tracking
Audit Trail
Timeline
```

### Completion Criteria

```
Tasks can be created
Tasks persist
Events are stored
```

---

# 7. Phase 3 — LLM Layer

## Goal

Integrate Planner, Reasoner, and Verifier.

---

# 7.1 LLM Gateway Module

Responsibilities:

```
Routing
Retries
Rate Limits
Fallbacks
Monitoring
```

---

# 7.2 Planner Module

Model:

```
Kimi K2.6
```

Responsibilities:

```
Goal Analysis
Planning
Task Breakdown
Replanning
```

---

# 7.3 Reasoner Module

Model:

```
Qwen3 Coder 480B
```

Responsibilities:

```
Code Analysis
Patch Generation
Architecture Understanding
```

---

# 7.4 Verifier Module

Model:

```
Nemotron 3 Ultra
```

Responsibilities:

```
Review
Validation
Regression Detection
```

---

## Completion Criteria

```
Planner Generates Plans
Reasoner Generates Findings
Verifier Reviews Outputs
```

---

# 8. Phase 4 — Tool System

## Goal

Create execution capabilities.

---

## 8.1 Tool Registry

Responsibilities:

```
Tool Discovery
Tool Registration
Capability Tracking
```

---

## 8.2 Tool Execution Engine

Responsibilities:

```
Execution
Monitoring
Timeouts
Retries
```

---

## 8.3 Terminal Module

Capabilities:

```
Command Execution
Process Management
Streaming
```

---

## 8.4 Browser Module

Capabilities:

```
Navigation
DOM Analysis
Screenshots
Automation
```

---

# 8.5 Filesystem Module

Capabilities:

```
Read Files
Write Files
Search Files
Move Files
```

---

# 8.6 Git Module

Capabilities:

```
Commit
Branch
Diff
Status
```

---

## Completion Criteria

```
Tools Executable
Streaming Works
Tool Results Stored
```

---

# 9. Phase 5 — Memory System

## Goal

Persistent intelligence.

---

# 9.1 Working Memory

Stores:

```
Current Task Context
```

---

## 9.2 Session Memory

Stores:

```
Current Session Knowledge
```

---

## 9.3 Project Memory

Stores:

```
Repository Knowledge
Architecture Knowledge
```

---

## 9.4 Experience Memory

Stores:

```
Past Fixes
Past Failures
Strategies
```

---

### Completion Criteria

```
Store Memory
Retrieve Memory
Semantic Search Works
```

---

# 10. Phase 6 — Repository Intelligence

### Goal

Enable large-scale repository understanding.

---

## 10.1 Repository Indexer

Responsibilities:

```
Parse Repository
Generate Metadata
```

---

## 10.2 Symbol Extractor

Extract:

```
Functions
Classes
Methods
Imports
```

---

## 10.3 Dependency Analyzer

Generate:

```
Dependency Graph
```

---

## 10.4 Call Graph Generator

Generate:

```
Function Relationships
```

---

## 10.5 Semantic Search Engine

Capabilities:

```
Code Search
Architecture Search
Reference Search
```

---

### Completion Criteria

```
Repository Search Works
Graphs Generated
Context Retrieval Works
```

---

# 11. Phase 7 — Agent Execution Engine

### Goal

Enable long-horizon autonomous workflows.

---

# 11.1 Orchestrator

Responsibilities:

```
Task Management
State Management
Agent Coordination
```

---

# 11.2 Execution Controller

Responsibilities:

```
Action Selection
Execution Tracking
```

---

# 11.3 State Machine

States:

```
CREATED
PLANNING
ANALYZING
EXECUTING
VERIFYING
COMPLETED
```

---

# 11.4 Checkpoint Engine

Responsibilities:

```
Save State
Restore State
Recovery
```

---

## Completion Criteria

```
Tasks Execute End-to-End
Checkpoint Recovery Works
```

---

# 12. Phase 8 — Antigravity IDE

## Goal

Build the operational workspace.

---

# 12.1 Chat Workspace

Displays:

```
Goals
Plans
Agent Communication
```

---

## 12.2 Browser Workspace

Displays:

```
Live Browser
DOM Events
Screenshots
```

---

## 12.3 Source Code Workspace

Displays:

```
Files
Diffs
Repository Search
```

---

## 12.4 Terminal Workspace

Displays:

```
Live Commands
Streaming Output
```

---

## 12.5 Memory Workspace

Displays:

```
Retrieved Context
Experience Memory
```

---

## 12.6 Timeline Workspace

Displays:

```
Events
Execution Progress
```

---

### Completion Criteria

```
All Workspaces Functional
Realtime Sync Operational
```

---

# 13. Phase 9 — Production Hardening

## Goal

Prepare for real-world usage.

---

# 13.1 Security Layer

Add:

```
Rate Limiting
Audit Logs
RLS Policies
Secret Scanning
```

---

# 13.2 Monitoring Layer

Add:

```
Metrics
Tracing
Logging
Dashboards
```

---

# 13.3 Reliability Layer

Add:

```
Retries
Fallbacks
Circuit Breakers
Recovery
```

---

# 13.4 Performance Optimization

Improve:

```
Retrieval Speed
Response Time
Execution Throughput
```

---

## Completion Criteria

```
Stable Production Deployment
```

---

# 14. Module Dependency Graph

```
Foundation
    ↓
Authentication
    ↓
Core Backend
    ↓
LLM Layer
    ↓
Tool System
    ↓
Memory System
    ↓
Repository Intelligence
    ↓
Agent Execution Engine
    ↓
Antigravity IDE
    ↓
Production Hardening
```

---

# 15. Recommended MVP Scope

To reach a working MVP quickly:

Build only:

```
Authentication
Projects
Tasks
Planner
Reasoner
Verifier
Terminal
Filesystem
Repository Search
Basic Memory
Chat Workspace
Source Code Workspace
Terminal Workspace
```

---

Do NOT initially build:

```
Advanced Collaboration
Enterprise Features
Knowledge Graph
Multi-User Workspaces
```

---

# 16. Recommended Team Allocation

## Backend

Focus:

```
APIs
Database
Agents
Execution Engine
```

---

### Frontend

Focus:

```
Antigravity IDE
Realtime UI
Workspace Integration
```

---

### AI Engineering

Focus:

```
Prompts
Retrieval
Reasoning
Verification
```

---

## 17. Risk Analysis

### Highest Risk Modules

```
Repository Intelligence
Long-Horizon Execution
Memory Retrieval
Verification
```

---

### Medium Risk Modules

```
Browser Automation
Realtime Infrastructure
```

---

### Low Risk Modules

```
Authentication
CRUD APIs
Storage
```

# 18. Success Milestones

## Milestone 1

```
User Creates Task
Planner Generates Plan
```

## Milestone 2

```
Reasoner Analyzes Repository
```

## Milestone 3

```
Verifier Reviews Findings
```

## Milestone 4

```
Tool Execution Works
```

## Milestone 5

```
Agent Completes Full Task
```

## Milestone 6

```
Antigravity IDE Fully Operational
```

# 19. Final Implementation Sequence

```
Phase 0
Foundation
        ↓
Phase 1
Authentication
        ↓
Phase 2
Core Backend
        ↓
Phase 3
LLM Layer
        ↓
Phase 4
Tool System
        ↓
Phase 5
Memory System
        ↓
Phase 6
Repository Intelligence
        ↓
Phase 7
Execution Engine
        ↓
Phase 8
Antigravity IDE
        ↓
Phase 9
Production Hardening
```

---

# 20. Final Summary

Agent Phantom Recovery should be implemented as a layered execution platform rather than a traditional AI application.

Each phase introduces a new capability while minimizing complexity and integration risk.

This roadmap ensures that:

```
Planning
+
Reasoning
+
```

```
Verification
+
Tools
+
Memory
+
Repository Intelligence
+
Execution
```

are built incrementally and validated before moving to the next stage, resulting in a stable, scalable, and production-grade autonomous engineering system optimized for Antigravity IDE.

---

# End of Document

Status: Approved

Version: 1.0

Document Type: Production Module Implementation Plan